1 WHAT IS COMPUTATIONAL THINKING?

OBJECTIVES

- Define computational thinking.
- Show how it can be used in different fields.
- Explain the current limitations of computational thinking.

WHAT IS COMPUTATIONAL THINKING?

Answering this question is actually quite challenging. Proponents of computational thinking (CT) have until very recently spent a lot of time debating over how to define it.

As recently as 2011, a workshop was organised where numerous individuals came together to explore what the nature of CT should be. Some at this workshop argued in favour of a rigorous and consistent definition (Committee for the Workshops on Computational Thinking, 2011). Conversely, others have argued that trying to **strictly** define CT is unnecessary (Voogt et al., 2015).

In the latter's view, understanding CT should not be done by coming up with a definition in the usual sense (in other words, creating a list of conditions that something must meet before being considered a match). Voogt et al. argue that defining CT shares similar difficulties with defining what a 'game' is. (Must every game pit at least one player against another? Does every game have the concept of winning? Should every game include some element of luck or randomness?) Like our understanding of a game, they say, the approach to defining CT should be fuzzier and considered as a series of similarities and relationships that criss-cross and overlap.

Other reasons exist to make defining CT a challenging endeavour. First, computational thinking is strongly related to computer science (CS), which itself can be problematic to define satisfactorily. Like CS, CT includes a range of both abstract and concrete ideas. They both share a universal applicability, and this broadness, while making it powerful, also makes CT hard to define concisely.

CT is also an idea that's both new and old. It's new in the sense that the subject suddenly became a hotly debated topic in 2006 after Wing's talk (Wing, 2006). However, many of its core ideas have already been discussed for several decades, and along the way people have packaged them up in different ways. For example, as far back as 1980,

Seymour Papert of the Massachusetts Institute of Technology pioneered a technique he called 'procedural thinking' (Papert, 1980). It shared many ideas with what we now think of as CT. Using procedural thinking, Papert aimed to give students a method for solving problems using computers as tools. The idea was that students would learn how to create algorithmic solutions that a computer could then carry out; for this he used the Logo programming language.[4] Papert's writings have inspired much in CT, although CT has diverged from this original idea in some respects.

Nevertheless, during the 10 years following Wing's talk, a number of succinct definitions were attempted. A small sample of them features in Table 1.1. While they hint at similar ideas, there appears to be some diversity in what these people say. Perhaps Voogt et al. were right, and our best hope of understanding CT is to build up those overlapping, criss-crossing concepts. As luck would have it, this work was already done by Cynthia Selby (Selby, 2013), when she scoured the CT literature for concepts and divided them into two categories: concepts core to CT, and concepts that are somehow peripheral and so should be excluded from a definition.

Table 1.1 A list of definitions of computational thinking

Definition	Source
'Computational thinking is the thought processes involved in formulating a problem and expressing its solution(s) in such a way that a computer—human or machine—can effectively carry out.'	(Wing, 2014)
'The mental activity for abstracting problems and formulating solutions that can be automated.'	(Yadav et al., 2014)
'The process of recognising aspects of computation in the world that surrounds us, and applying tools and techniques from Computer Science to understand and reason about both natural and artificial systems and processes.'	(Furber, 2012)
'A mental orientation to formulating problems as conversions of some input to an output and looking for algorithms to perform the conversions. Today the term has been expanded to include thinking with many levels of abstractions, use of mathematics to develop algorithms, and examining how well a solution scales across different sizes of problems.'	(Denning, 2009)
'[Teaching CT is teaching] how to think like an economist, a physicist, an artist, and to understand how to use computation to solve their problems, to create, and to discover new questions that can fruitfully be explored.'	(Hemmendinger, 2010)

In this book, I will follow a very similar approach to Selby. Certain concepts are considered as 'core' to CT and are covered in great detail. Others are included, but treated as peripheral and covered in much less detail.

This book considers the core concepts of CT to be:[5]

- logical thinking;
- algorithmic thinking;
- decomposition;
- generalisation and pattern recognition;
- modelling;
- abstraction;
- evaluation.

Other peripheral concepts will be mentioned, but not treated as essential to the topic of CT. They include:

- data representation;
- critical thinking;
- computer science;
- automation;
- simulation/visualisation.

I will define individual concepts as they are introduced over the course of the book.

HOW IS COMPUTATIONAL THINKING USED?

CT can be applied by anyone who is attempting to solve a problem and have a computer play a role in the solution. To give us some idea of how CT is used, not just in computer science but in a range of subject areas, we can look at examples (sourced from Barr and Stephenson (2011)) of the core computational thinking concepts just listed.

For example, what could algorithmic thinking mean in different situations? To a computer scientist, it means the study of algorithms and their application to different problems. To a mathematician, it might mean carrying out long division factoring or doing carries in addition or subtraction. A scientist might think of it as the process of doing an experimental procedure.

Similarly, abstraction has application beyond the computer scientist's view of it. When a linguist uses simile and metaphor, or writes a story with branches, they're using abstraction, as are social scientists who summarise facts and use them to draw conclusions. When a scientist builds a model or a mathematician uses algebra, they too have introduced abstraction into their work.

The following are examples from other sources discussing specific examples of CT in action.

Example: Pipelining a graduation ceremony

> Dean Randy Bryant was pondering how to make the diploma ceremony at commencement go faster. By careful placement of where individuals stood, he designed an efficient pipeline so that upon the reading of each graduate's name and honors by Assistant Dean Mark Stehlik, each person could receive his or her diploma, then get a handshake or hug from Mark, and then get his or her picture taken. This pipeline allowed a steady stream of students to march across the stage (though a pipeline stall occurred whenever the graduate's cap would topple while getting hug from Mark).
>
> (Wing, 2011)

Example: Predicting climate change

> Predicting global climate change is only possible because of advanced computer models. According to the UK Met Office, 'The only way to predict the day-to-day weather and changes to the climate over longer timescales is to use computer models.'
>
> (Furber, 2012)

Example: Sorting music charts

> I showed up to a big band gig, and the band leader passed out books with maybe 200 unordered charts and a set list with about 40 titles we were supposed to get out and place in order, ready to play. Everyone else started searching through the stack, pulling out charts one-at-a-time. I decided to sort the 200 charts alphabetically $O(N \log N)^6$ and then pull the charts $O(M \log(N))$. I was still sorting when other band members were halfway through their charts, and I started to get some funny looks, but in the end, I finished first. That's computational thinking.
>
> (Roger Dannenberg in Wing, 2011)

Example: Assisting police, lawyers and judges

> Computational Thinking has a long tradition in influencing the law, especially in the dream of providing a set of logical rules that can automate the process of reaching a verdict, [underpinning] its desire to minimise human discretion and maximise predictability of outcome... legal reasoning systems have been making inroads where they merely try to assist those making legal decisions. For instance, researchers at the Joseph Bell Centre have built a system that constructs a space of hypotheses to explain the evidence in a crime scene. This has been used to remind detectives of hypotheses they might otherwise have missed.
>
> (Bundy, 2007)

DISCLAIMERS

This section discusses common misunderstandings about CT, as well as its current shortcomings.

What is computational thinking not?

There is bound to be some confusion over what CT is and is not. For one thing, it's closely related to other subjects, like computer science and programming, and distinguishing between them may present difficulties. For another, its definition is broad and has been subject to debate for several years. This section will discuss something that some people might have been led to believe about CT which isn't necessarily the case. In the process, more details about CT's nature will be revealed.

First, teaching computational thinking is not the same as teaching computer science. The main aim of teaching the latter is to educate students in the study and application of the principles of mathematical computation. Perhaps to counter this impression, some, including Jeannette Wing, have said that computational thinking is 'thinking like a computer scientist'. However, others have criticised this phrase (for example, Denning, 2009; Hemmendinger, 2010) because it leaves in place a strong association with computer science, which runs the risk of people seeing it not as an everyday skill, but merely as a repackaging of CS.

Teaching programming (a subfield of CS) is mainly done to educate students in how best to write programs, and it focuses on the production of high-quality software. CT, while sharing some aspects with these other subjects, is better described as an approach to problem-solving. What makes it distinct from other problem-solving approaches is that CT assumes a computer will execute the eventual solution.



CT teaches an approach to problem-solving where the ultimate aim is to provide a solution whose form means it is ready to be programmed into a computer.

CT takes a relatively small subset of concepts – which just happen to be important to CS – and uses them to construct a widely applicable, problem-solving approach.

These distinctions are important. Teaching computing skills as a mandatory part of the school syllabus has already been attempted in many places around the world. While these endeavours are noble and worthwhile, evidence is inconclusive about whether or not they succeed in their aim of teaching **transferable** skills (Voogt et al., 2015); that is to say, skills that students can intuitively take with them into various domains. CT attempts to perform better in this regard. By distilling some of the core aspects of CS that are relevant in many other fields, CT can be a course in the core school syllabus that is relevant to everyone. In the UK computing curriculum for example, CT already plays a big role (Department for Education, 2013). It can even be part of a CS or programming course that focuses exclusively on computer-based problem-solving.

In stressing the differences between computer science and computational thinking, it's worth pointing out that CT concepts are hardly exclusive to CS. As David Hemmendinger

(2010) points out, 'Constructing models, finding and correcting errors, drawing diagrams, analyzing – these are all parts of scientific (and other) activities.' Natural scientists, for example, have long been advocating that computation is a 'third leg' of the scientific process (alongside theory and experimentation) and that computational thinking is essential to their work (Denning, 2009).

> [The] ultimate goal should not be to teach everyone to think like a computer scientist, but rather to teach them to apply these common elements to solve problems and discover new questions that can be explored within and across all disciplines.
>
> (Barr and Stephenson, 2011)

Current shortcomings

We'll very soon proceed to consider the details of computational thinking. Before that happens, however, an honest discussion of the subject in its current form should be up front with any shortcomings. Keep in mind that these may disappear as CT matures.

Maturity

Although it has influences and predecessors dating back several decades, CT as a formal concept is still relatively immature. Workshops were organised to explore the meaning of CT only five years prior to this book being written. As little as three years ago, Cynthia Selby wrote an important paper proposing a definition for CT (Selby, 2013), because definitions at the time were still vague. Most other disciplines have long since stopped organising gatherings to decide what exactly it is they do. Nevertheless, a consensus is now emerging on what to include in a definition, largely in line with Selby's proposal of CT as 'a focused approach to problem solving, incorporating thought processes that utilize abstraction, decomposition, algorithms, evaluation, and generalizations' (2013). ('What is computational thinking?' on p. 7.)

Efficacy

A further consequence of CT's youth is the relative paucity of evidence for how effective teaching CT actually is. It has so far not been taught widely for an extended time. Nevertheless, the body of evidence will grow as studies into its efficacy are conducted. Experience reports and testimonials provide a more immediate form of evidence.

Perceived imperialism

Other commentators have cautioned that proponents of CT risk being imperialistic and off-putting. Their claims like 'you should think like a computer scientist' or 'computational thinking is X' (where X is something they approve of) appear territorial. As David Hemmendinger (2010) said, proponents should be careful that CT doesn't get into the habit of saying, 'If it's a good way of thinking, then it's ours.' It should be noted that many practices in CT are not original to the subject. In fact, they're not all original to computer science; many of them have been used in other fields as well as everyday life for a long time. They do not simply become computational by virtue of finding application in computing.

However, it does somewhat mitigate CT's immaturity when you realise that its constituent parts (such as logic, algorithms and decomposition) are individually tried, tested and mature.

SUMMARY

Computational thinking is an approach to problem-solving that involves using a set of practices and principles from computer science to formulate a solution that's executable by a computer. It's not just for programmers. In fact, it's applicable in a diverse array of fields.

EXERCISES

EXERCISE 1

List the core concepts of CT.

EXERCISE 2

Give an example of how you think people in each of the following occupations think computationally:

A. mathematician;
B. scientist;
C. engineer;
D. linguist.

EXERCISE 3

Think of everyday activities in which you participate that involve computational thinking.

2 LOGICAL AND ALGORITHMIC THINKING

OBJECTIVES

- Learn the importance of logic to computational thinking.
- Appreciate the difference between deductive and inductive reasoning.
- Understand Boolean logic and its importance to computation.
- See the importance of using logical and mathematical notation instead of natural language.
- Learn the properties of algorithms: sequence, iteration, selection.
- Understand the importance of state in algorithms.
- See common mistakes made in logical and algorithmic thinking and learn how to avoid them.

APPROACH

Logic and algorithms are essential to CT. They underpin the subject and rear their heads repeatedly throughout its application. The good news is: humans already have an innate, intuitive understanding of both logic and algorithms. The bad news is: they are both mathematical concepts in nature. Consequently, each has its own set of rules, procedures and definitions, which are very precise and systematic. That means you can't rely solely on your intuition when dealing with these topics, otherwise you'll make mistakes.

The best way to overcome this is to learn the precise, yet difficult core concepts. One could write a whole book just on logic or algorithms (many already have), but such a comprehensive treatment lies far out of scope for this volume, where they are just one topic of many. Consequently, this chapter must narrow its focus.

Research exists that shows us where newcomers tend to make mistakes (Pane et al., 2001), which will allow this chapter to concentrate on the more troublesome spots. I will focus on just a few core elements relevant to getting you into the habit of thinking logically and algorithmically. By the end of the chapter, you will have learned how to apply logic and algorithms to problem-solving. With some practice, they should become second nature.

LOGICAL THINKING

This section shows how important logic is to computational thinking. It introduces logical reasoning, along with Boolean and symbolic logic, and shows how to turn intuitive ideas into mathematically sound, logical expressions.

In brief: Logic

Put simply, logic is a system used for distinguishing between correct and incorrect arguments. By 'argument', I'm not referring to two people in a shouting match. I mean the philosophical idea of an argument; namely a chain of reasoning that ends up in a conclusion.

Logic includes a set of principles that, when applied to arguments, allow us to demonstrate what is true. You need no special training to begin doing this, as this classic introductory example of a logical argument demonstrates:

1. Socrates is a man.
2. All men are mortal.
3. Therefore, Socrates is mortal.

Even those of us without philosophy degrees understand this argument. We perform this particular brand of reasoning all the time (there's a draught in the room; a window is open; therefore, the draught is coming from the window). However, we don't always carry it out **correctly**, which can lead us to form wrong conclusions. And since we use computers essentially to automate our reasoning, we must learn to perform logic correctly before writing a computer solution.

In a sense, applying logic is a way of developing and testing a hypothesis. Using this way of thinking, applying logic assumes you already know at least some things for sure and allows you to use that knowledge to arrive at some further conclusions.

In a logical argument, each individual thing you already know (or assume) is called a **premise**. A premise is like any ordinary statement you or I might make, except that it can be evaluated to obtain an answer of 'true' or 'false'. A premise, therefore, has a **truth value**. In the Socrates example, our premises fit this requirement. It is either true or not that all men are mortal. The same goes for whether Socrates is a man or not. Other forms of expression, like questions or commands, can't be premises to an argument. It's neither true nor false to say 'Break a leg!' or ask 'What time is it?', for example.

Once all the premises are stated, the next step is to analyse them and react accordingly with a conclusion. Most of the magic lies in this step and this is what we will focus on.

Inductive vs deductive arguments

It's important to realise that some logical arguments are stronger than others. In fact, you can categorise arguments based on their certainty. The two best-known categories are **deductive** and **inductive**.

A deductive argument is the strongest form of reasoning because its conclusion necessarily follows from its premises (so long as it has been constructed properly and the premises are incontrovertibly true). We've already seen an example of deductive reasoning: the assessment of Socrates's mortality. While deductive arguments are strong, they have very strict standards, which makes them hard to construct. A deductive argument can fail in one of two ways.[7]

First, one of its premises could turn out to be false. For example:

1. Missie is a dog.
2. All dogs are brown.
3. Therefore, Missie is brown.

Premise 2 is false: not all dogs are brown. Even though the argument follows the exact same form as the Socrates example, it fails because at least one of its premises is false. Any argument with false premises fails. In computer jargon, this is an example of 'garbage in, garbage out'.



You can express the form of an argument by substituting symbols for objects. This helps when trying to work out if your reasoning is valid. The form of the Socrates example is: 'A is a B; All Bs are C; Therefore, A is C.' We'll see later how to handle logic symbolically.

The second way a deductive argument fails is when the conclusion doesn't necessarily follow from the premises. For example:

1. All tennis balls are round.
2. The Earth is round.
3. Therefore, the Earth is a tennis ball.

This argument fails because of faulty logic. Yes, all tennis balls are round, but so are lots of other things. In symbolic terms, this argument follows the form, 'All As are B; C is B; Therefore C is an A', but since this is in an invalid form, the argument is automatically invalid too.



Many books and online resources explain faulty forms of reasoning (i.e. fallacies). A good starting point is *Attacking Faulty Reasoning*, by T. Edward Damer (2005).

In reality, deductive arguments are relatively rare. You'll usually encounter them only when the knowledge you're dealing with is nice and neat. However, the real world often presents us with knowledge that's patchy, messy or provisional. Real-life issues are often nuanced (despite what you might read on social media). For this, we have inductive reasoning, which deals in probabilities rather than hard, black and white rules.

The premises in an inductive argument are not unquestionably true. Rather, we have some level of confidence in them. The form of the argument doesn't guarantee that the conclusion is true, but it probably results in a trustworthy conclusion. For example:

1. A bag contains 99 red balls and one black ball.
2. 100 people each drew one ball from the bag.
3. Sarah is one of those 100 people.
4. Therefore, Sarah probably drew a red ball.

This is a perfectly fine inductive argument, so long as you acknowledge the aspect of probability involved.

These aspects of reasoning are important in CT because computers are involved. The answer a computer gives is only as reliable as its reasoning, and since a computer is automating your reasoning, it's your responsibility to make sure:

1. that reasoning is valid;
2. you give the computer reliable input;
3. you know how to interpret what conclusion the computer reports, that is, is the result unquestionably true (the reasoning was deductive) or probably true (the reasoning was inductive)?

Boolean logic

Even though much of our reasoning is inductive, computers are not well equipped to deal with shades of grey. Their binary nature makes them more apt to deal with black and white issues. In order to instruct computers to make logical decisions, we need a system of logic that maps well onto this way of thinking. Boolean logic is such a method. It's a form of logic that deals with statements having one of only two values: true or false (usually). Different corresponding values could be used in other contexts: 1 or 0 for example, on or off, black or white.

Propositions

Statements in Boolean logic are also known as **propositions**, which have several basic properties.

First, a proposition can only have one value at any one time. In other words, a single proposition can't be both true and false simultaneously. There is no way to express levels of certainty. True means true; false means false. Consequently, you should keep in mind what was said earlier about deductive and inductive arguments: whereas real-life problems often present us with probabilities, the basic Boolean world deals in certainties. Some of your efforts will go into mapping real-world, grey areas onto Boolean black and white.

Second, propositions must have clear and unambiguous meaning. For example, a statement like: 'It is travelling fast', can certainly be evaluated as either true or false. However, it's ambiguous as stated. If 'it' is a car travelling at 150 mph along the

motorway, that's certainly fast. But if 'it' is a spacecraft travelling towards Mars at 150 mph, that's undoubtedly slow.



Qualify statements where necessary to avoid ambiguity. For example, you could restate the example above as: 'It is travelling fast, where "fast" is 70 mph or greater.'

Third, it's possible to combine individual propositions to make more complex ones (called **compound** propositions). For example, 'Jenny is wearing the shirt and the shirt is red.' This is helpful because we often want to evaluate several statements before reaching a conclusion. We make compound propositions by connecting single propositions together using **logical operators**.

Logical operators

Imagine you say, 'If the weather is sunny and I'm on holiday, then I'm going to lie in the garden.' You've just used logical operators. That statement contains two propositions, which serve as conditions for whether, or not, you decide to lounge on the grass:

1. The weather is sunny.
2. You're on holiday.

If both are true, then you can lie in the garden. If **either** of them are false (say, the weather is lousy or you have to go to work), then sunning yourself is not an option. That's because you joined them using the logical operator **AND**, which demands that both propositions should be true for the conclusion to be true.

Many different logical operators exist. It's worth looking at the most important ones in more detail because, even though we use them daily in informal speech, they have specific meanings in logic that occasionally run counter to our intuitive understanding.

To provide illustrative examples, we'll use the operators to describe the rules of a simple game: Noughts and Crosses.[8]

AND: the technical name for this operator is **conjunction**. (We just saw an example of a conjunction when we reasoned about whether to lie in the garden.) It chains propositions together in a way that **all** of them must be true for the conclusion to be true. If any of them are false, the conclusion is rendered false also. In classical logical arguments like we've seen so far, the presence of AND between propositions is implicit, but we can (and should) include them explicitly. So, for example:

1. At least one square on the board is still empty.
2. Neither player has achieved a row.
3. Therefore, the game is still in progress.

This can be expressed as:

> If at least one square on the board is still empty and neither player has achieved a row, then the game is still in progress.

OR: the technical name for this operator is **disjunction**. This operator chains propositions together in a way that **at least one** of them must be true for the conclusion to be true also. The only way that the conclusion is falsified is if all propositions are false. For example:

> If player 1 achieves a row or player 2 achieves a row, then the game is over.

In this case, only one condition needs to be true to end the game. In fact, due to the nature of the game, a maximum of one of these conditions can be true simultaneously. OR can also be used in cases when both conditions can be true at the same time:

> If a player achieves a row or all squares become occupied, then the game is over.

NOT: the technical name for this operator is **negation**. This operator doesn't chain propositions together itself, rather it modifies a single proposition. Specifically, it flips the truth value. Sometimes, negating a proposition can make it easier to express the chain of reasoning. For example:

> If a square is not occupied, then a player may add their symbol to that square.

IMPLIES: the technical name for this operator is **implication**. Using this operator is to state that there is a correlation between the two statements. If the first statement is true, then the second must be true also. Keep in mind, this is a correlation **not** a causation. Therefore, you can't necessarily work backwards from the conclusion of an implication. Consider this:

> If a player achieves a row, then the game is over.

It's true that a game ends when a player achieves a row, but it's not the only way to end a game. Saying 'the game is over, therefore a player achieved a row' is not always true.[9] The game could be over because it ended in a draw.

IF AND ONLY IF: the technical name for this operator is **biconditional**. This behaves very similarly to implication, but a biconditional means that the second proposition is influenced **solely** by the first. If the first is true, the second is true. If the first is false, the second is false. No exceptions. For example:

> If and only if all squares are occupied, then no more moves are possible.

In this case, we **can** work backwards. The only reason no more moves are possible is because all squares are occupied.

Symbolic logic

Logic requires precision and an absence of ambiguity, but it can be difficult to meet these requirements when using natural language. Not only can statements grow wordy and confusing, but their meaning can become almost unavoidably ambiguous.

Saying 'she's a fast reader or she's read the book before' raises the question: could both be true? I would say so. But what about 'Starter is a soup or a salad'? In this case, both can't be true. Notice, however, that in each case we used the word 'or' to join two propositions, but its use implied different meanings in both cases. The other logical operators can similarly be misused to mean different things.

To help us manage our reasoning, mathematics gives us symbolic logic, which recommends using symbols instead of natural language sentences. The earlier example, 'If at least one square on the board is still empty and neither player has achieved a row...' contains two propositions that are replaceable with symbols. If,

P = at least one square on the board is still empty

Q = neither player has achieved a row

S = the game is still in progress

then we can say:

If P and Q, then S

Not only does that reduce the clutter, but it becomes more intuitive to treat each proposition as a variable. After all, each proposition has a value that can be true or false at different times.

The operators, too, are often replaced by symbols (see Table 2.1).

Table 2.1 Logical operators and their symbols

Operator name	Symbol	Example
AND	$\wedge$	$A \wedge B$
OR	$\vee$	$A \vee B$
NOT	$\neg$	$\neg A$
IMPLIES	$\rightarrow$	$A \rightarrow B$
IF AND ONLY IF	$\leftrightarrow$	$A \leftrightarrow B$

Our example can then further be reduced to:

$$P \wedge Q \leftrightarrow S$$

(Rather than overload you with symbols just now, I'll continue to use the operator name rather than the symbol. When you see the operator name in upper case letters, it specifically means the logical operator.)

Symbolic logic also gives each logical operator formal rules, which promote precision and eliminate ambiguity. The meaning of each logical operator is specified in precise mathematical detail. The standard way to do this is to use a truth table. This lists all the possible combinations of values of the propositions. The truth table states whether each combination is logically valid or not. Let's start by looking at the truth table for conjunctions.

To read a truth table, take it one line at a time. The first line of Table 2.2 says that if both propositions are individually valid by themselves, then it's valid to say they are both true together. In the three other eventualities, it is invalid to say they are both true together because one or both propositions are false.

Table 2.2 Truth table of conjunction (logical AND)

P	*Q*	*P* AND *Q*
True	True	True
True	False	False
False	True	False
False	False	False

Keep in mind that the use of 'True' and 'False' can be replaced by other pairs of opposing symbols. Table 2.3 shows the same information as Table 2.2, except that it uses 1 and 0 instead of True and False.

Table 2.3 Truth table of conjunction (using 1 and 0 symbols)

P	*Q*	*P* AND *Q*
0	0	0
0	1	0
1	0	0
1	1	1

Figure 2.1 is a Venn diagram depicting the behaviour of conjunctions.

Figure 2.1 Venn diagram for the AND operator

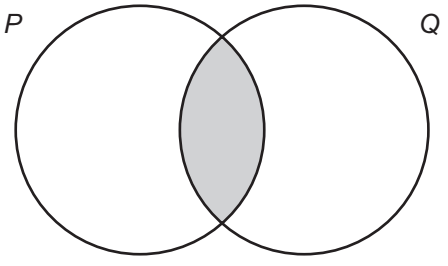


Table 2.4 Truth table of disjunction (logical OR)

P	*Q*	*P* OR *Q*
True	True	True
True	False	True
False	True	True
False	False	False

Table 2.4 shows that it is valid to consider a disjunction true so long as one or more of its propositions are true. That means the intention behind the earlier example – 'A starter can be soup or a salad [but not both]' – doesn't align with this logical meaning of OR (see Figure 2.2).[10]

Table 2.5 Truth table of negation (logical NOT)

P	NOT *P*
True	False
False	True

Figure 2.2 Venn diagram for the OR operator

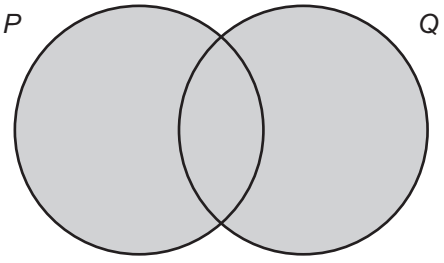


Figure 2.3 pictorially represents the difference between a proposition with and without the NOT operator. In other words, whatever is true becomes false, and whatever is false becomes true.

Figure 2.3 Venn diagram for the NOT operator

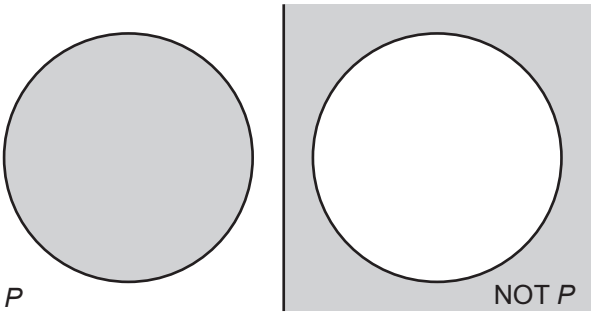


Table 2.6 Truth table of implication (logical IMPLIES)

P	*Q*	*P* IMPLIES *Q*
True	True	True
True	False	False
False	True	True
False	False	True

The first two lines of Table 2.6 seem sensible. If *P* being true implies *Q* is true and both are indeed true, then the implication is valid. Likewise, if *P* is true, but its consequent is not true, then the implication is invalid. The last two lines, however, seem odd. The implication is valid regardless of the value of *P*.



Why is it valid for *P* to imply *Q* when *P* is false? In '*P* implies *Q*', *P* is known as the antecedent and *Q* as the consequent. The explanation is that no conclusion can be drawn about an implication when the antecedent (i.e. *P*) is false. Consider the statement 'if it is raining (*P*), then the grass is wet (*Q*)'. Saying 'it is not raining' (NOT *P*) doesn't contradict that the grass is wet. The grass could be wet for another reason, such as because the sprinkler is on. Since there is no contradiction, mathematicians have judged that an implication with a false antecedent is true until proven otherwise (see Figure 2.4).

Figure 2.4 Venn diagram for the IMPLIES operator

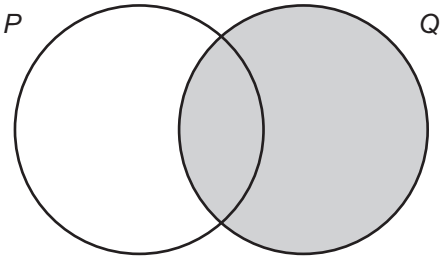
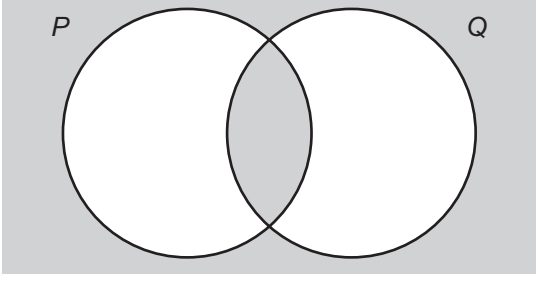


Table 2.7 Truth table of biconditional (logical IF AND ONLY IF)

P	*Q*	*Q* IF AND ONLY IF *P*
True	True	True
True	False	False
False	True	False
False	False	True

Figure 2.5 depicts the IF AND ONLY IF operator. Only when values agree can the statement be considered valid. *P* and *Q* can agree when their values overlap, either by being both true or both false.

Figure 2.5 Venn diagram for the IF AND ONLY IF operator



ALGORITHMIC THINKING

We now move from logical thinking on to algorithmic thinking. This section will introduce the basic properties of algorithms (sequence, iteration and selection), as well as the use of state.

In brief: Algorithms

Logic and algorithms are not the same. Rather, algorithms build on logic because, as part of their work, they make logical decisions. The other part of their work is 'stitching' those decisions together.[11]

Logic gives you a set of rules that allow you to reason about some aspect of the world. That world does not have to be static. Logic can deal with things that are dynamic and continually changing. This we saw with the Noughts and Crosses examples. The rules of the game were laid out as logical statements. Sometimes the propositions were true, sometimes false. By working out their truth values, we could come to conclusions about how the game would be in different situations.

But if we want to go further, if we want to build functioning systems based on rules, then logic alone isn't sufficient. We need something that can integrate all these rules and execute actions based on the outcomes of evaluating them. That something is algorithms, and they are the power behind all real-world computational systems.



Algorithm: a sequence of clearly defined steps that describe a process to follow a finite set of unambiguous instructions with clear start and end points.

Algorithms are a way of specifying a multi-step task, and are especially useful when we wish to explain to a third party (be it human or machine) how to carry out steps with extreme precision. As with logic, humans already have an intuitive understanding of algorithms. But, at the same time, a rich and precise science dictates exactly how algorithms work. Gaining a deeper understanding of this will improve your algorithmic thinking. This is important because a correct algorithm is the ultimate basis of any computer-based solution.

Intuition vs precision

Algorithms are tricky things. In a sense, they're intuitive concepts that have been with us for centuries, yet they only received a definition in the last hundred years or so (Knuth, 1997). It's easy to articulate them intuitively, but at the same time computer science's concept of an algorithm is multi-faceted and can take beginners some time to comprehend. Misunderstandings commonly arise (see Pea et al., 1987; Pane et al., 2001), so we'll focus on building a solid understanding.

Before going into the hard details, let's take as our starting point the simple definition in the box above. You've very likely dealt with algorithms in this sense already. Anyone who has followed a recipe while cooking has encountered something like this. Or anyone

who went on a treasure hunt as a child. Or anyone who has assembled furniture. An algorithm is a way of making our processes explicit so that you can communicate them to someone else in a way that allows them to carry out those same steps too. These are only analogies however. Strictly speaking, algorithms perform operations on data rather than cake ingredients or coffee tables.

The earliest pioneers of computer science examined what it meant to communicate ideas to a computer. They sought a means for us to take the ideas in our heads and put them into a form that computers can understand and compute on our behalf. They discovered that our existing, intuitive ideas of algorithms were insufficient, so they took those ideas and re-formed them into the kind of clear and unambiguous form that computers require. By doing so, they made algorithms our means for giving computers instructions.

As precise as they became, algorithms were also rendered more complex, so much so that a formal description of them can stretch over several paragraphs. In our case, the best way to approach a definition is to take each property of algorithms, one by one, and explain it. Along the way, we'll use an analogy to one of our intuitive ideas of an algorithm (in this case, following a recipe) to help us understand.

Defining algorithms

The definition of an algorithm is complex and involves several properties. This subsection describes those properties.

Collection of individual steps

The first property to mention just restates something said earlier: an algorithm is a **collection of individual steps**. A recipe fits this analogy quite simply, filled as it is with steps like: 'pre-heat the oven to 180 degrees Celsius' or 'add two tablespoons of sugar to the bowl'.

Definiteness

Following on from that property is **definiteness**, meaning that every step must be precisely defined. Each step in an algorithm can have one and only one meaning, otherwise it is ambiguous. Similarly, chefs have come to the same conclusion, which is why they produce recipes using precise measurements instead of writing things like 'some sugar' or 'cook it for a while'.

Sequential

Algorithms are also **sequential**. The steps that make up the process must be carried out in the order specified. Failing to do this means that the result of executing the algorithm is likely incorrect. Think back to the analogy. Dicing an onion and frying an onion are different steps. Dicing an onion **before** you fry it has a different outcome than the reverse. Similarly, multiplying a number by 2 then adding 5 to it yields a different result from adding 5 first **then** doubling it. Like a recipe, you must respect the sequence when running through an algorithm for it to have any meaningful result.

Detour: State in algorithms

It's worth going on a brief detour here to examine why sequence is so important to algorithms. It's all to do with **state**, by which I simply mean the current values of all

the things the algorithm is keeping track of. As a computer progresses through an algorithm, just as you progress through a recipe, the state of things can change. Clearly sequencing the steps of an algorithm ensures the that state always changes in the same way whenever the algorithm is executed.



State: the current configuration of all information kept track of by a program at any one instant in time.

There's an important implication to this: there is **no 'global view'** when it comes to algorithms. This means that, at each instant in time, the environment in which the algorithm is being run exists in some particular state. But by the time the next step is executed, something might have changed. The environment really exists as a series of snapshots, one for each step of the algorithm. The recipe analogy spells this out. At the start you might have butter, flour, milk, eggs and sugar. After each step, you take a photograph of the kitchen. The photos will show that, bit by bit, the state of the ingredients changes. Flour goes into a bowl; then the eggs join it; then the butter goes into the pan; and so on. There is no global view of the ingredients; just a series of snapshots.

For algorithms, this means that individual steps are executed one by one and only a single step can be under consideration at any one time. Once a step has been executed, the computer forgets all about it and moves on to the next.

This means that some way of 'remembering' things that happened in previous steps needs to be provided. If you, the algorithm's author, want the computer to remember the result of a step for later use, you have to explicitly tell the computer to do so. Algorithms provide for this by means of **variables**.



Despite sharing the same name, variables in programming do not correspond exactly to variables from mathematics. Mathematical variables are used in a function to represent quantities that vary. A variable in an algorithm is more like a scratchpad that's used as a placeholder for important information the computer should keep note of. It can also have its values updated throughout an algorithm's execution (this is called **assignment**). For example, if your algorithm counts the number of times a specific word appears in a news article, it would use a variable to keep track of the number as it reads through the text.

Controlling algorithm execution

In this section we'll look at two ways of controlling the execution of an algorithm.

Iteration

As well as recording information pertinent to a problem, variables can also be used to control the execution of an algorithm. At a basic level, they can be used in two ways. One of those is **iteration**, also known as looping. Iteration allows you to repeat a series

of steps over and over, without the need to write out each individual step manually. This can be very useful. Imagine having to write out the lyrics to a song like *99 Bottles of Beer*. If you don't know it, it goes like this:

99 bottles of beer on the wall,
99 bottles of beer.
Take one down, pass it around,
98 bottles of beer on the wall.

98 bottles of beer on the wall,
98 bottles of beer.
Take one down, pass it around,
97 bottles of beer on the wall.

And keeps repeating until the last verse:

1 bottle of beer on the wall,
1 bottle of beer.
Take it down, pass it around,
No more bottles of beer on the wall.

To save yourself a lot of typing, you could, instead, write just one verse along with instructions on how to repeat all the following verses. Something like:

X stands for the number of bottles. At the start, X is 99. Sing the verse:

X bottles of beer on the wall,
X bottles of beer.
Take one down, pass it around,
X-1 bottles of beer on the wall.

Subtract 1 from X. Repeat the verse if X is greater than 0, otherwise finish.

This is an example of iteration in action and it shows two things:

1. How a variable is used to control algorithm execution. The thing that changes from one loop to the next is encapsulated in a variable, in this case the number of remaining beer bottles. Everything else stays constant.
2. You must specify under what conditions the loop should terminate.

This last point brings us to the second method used for controlling algorithm execution.

Selection

In a loop, one way to control how many times the steps are repeated is simply to specify it; something like 'repeat the following steps 99 times'. But notice that the example above doesn't do that. Instead, it uses *selection* (aka a conditional), which is a way to test a variable's current value and make a decision based on it. In the '99 Bottles' song, the conditional comes at the end (repeat the verse **if X is greater than 0**). It's telling the computer to do something so long as the condition specified is currently true. So long as

a positive number of bottles remain, that condition is true and so the verse is repeated. But we know that the number is steadily decreasing and will eventually reach zero. At this point the condition becomes false (zero is not greater than zero), and so the verse is not repeated again.

Conditions can be used at any stage in an algorithm, not only to control loops. Wherever they are, they all serve the same purpose: creating a point at which the computer has to decide between performing a set of steps or not, and encapsulating the information needed to make that decision.

Example algorithm

This example illustrates some of the ideas just discussed. It is a partial and very simple algorithm for controlling a game of Noughts and Crosses. It's written using **pseudocode**, which is an informal means of writing an algorithm. It's aimed at a human audience rather than a computer, but it nevertheless closely follows the conventions of programming languages. This means you can easily translate your ideas into real program code by first writing them as pseudocode.

```
 1. game is started
 2. begin loop:
    3. prompt player to choose a square
    4. if chosen square is not occupied, then put player's symbol in
       that square
    5. check board to see if a row has been achieved
    6. if a row has been achieved, then game is won
    7. if a row has not been achieved and no squares are available,
       then game is drawn
    8. switch to other player
 9. exit loop if game is won or game is drawn
10. display message 'Game over'
```

Let's examine this algorithm:

- When this algorithm is executed, the computer goes through each line one at a time. (This is an example of **sequence**.)
- Line 1 initialises a variable, `game`, with a particular value, `started` (**state**).
- Line 2 sets up the starting point of a loop (**iteration**). All lines that are 'inside' this loop are indented to make it clearer what's inside and outside.
- Line 3 asks for input from the player. The choice is recorded.
- Line 4 makes a **selection** based on the player's choice. (Incidentally, this is a rather user-unfriendly algorithm. If a player accidentally chooses an occupied square, they get no opportunity to fix their mistake!)
- Lines 6 and 7 both make selections, altering the value of the `game` variable under certain conditions.

- Line 9 is the end of the loop. A choice is made whether or not to go through the loop again depending on the associated condition **at this point**. There are two possibilities:

    1. The `game` variable hasn't been altered yet, meaning it still has the value `started`. In this case, execution moves back to line 3.
    2. At some point, the game was won or drawn. In either case, the loop ends and execution continues onto line 10, whereupon the game finishes with a traditional 'Game over' message.

'GOTCHAS'

When first encountering logic and algorithms, most people tend to make the same mistakes or faulty assumptions. This section will point them out and explain how to avoid them.

The need for clarity and meticulousness

You might have noticed the emphasis on clarity, precision and meticulousness when working with logic and algorithms. Computer programs demand precision in their rules and instructions to such an extent that it might have the appearance of pedantry. But it really isn't. There are good reasons behind the need for clarity, which all boil down to a computer's nature.

A couple of facts about this are worth pointing out:



- A computer will do exactly as it is told. If it is told to do something impossible, it will crash.
- A computer has no innate intelligence of its own. It will not do anything that it has not been instructed to do.
- A computer has no common sense. It will not try to interpret your instructions in different ways. It will not make assumptions or fill in the obvious blanks in an incomplete algorithm.

In short, a computer will do only what you tell it to do. If you don't tell it to do something, it won't do it. If you tell a computer to do something stupid, like divide a number by zero, it will try to do it anyway. If a computer does something you didn't think you told it to do, that's probably because you misunderstood your own instructions to it.

Earlier, I drew an analogy between an algorithm and a recipe. However, the audience for a recipe is another person. Granted, a person can be the audience for an algorithm, but people are used to communicating using natural language. Even when it's very formal, that language can still be ambiguous and sometimes open to interpretation. Furthermore, the author may leave certain things up to a reader's common sense rather than labour a point.

You must be mindful of the (electronic) computer's limitations outlined above. Think of giving a recipe to an exceedingly literal-minded robot from a cheesy sci-fi film. Would it be the robot's fault if it read 'stir the milk, flour and eggs in a bowl', then tried to climb into the bowl itself? Or if it waited forever for the 'dough to rise' because the dough hadn't begun to levitate yet?

The point is, the goal of problem-solving using computational thinking is a solution that a computer can execute. That means an algorithm with precise, unambiguous meaning that requires neither creative intelligence nor common sense to understand fully.

And that often means splitting hairs and spelling things out in seemingly pedantic detail.

Incorrect use of logical operators

Our intuitive understanding of language leads to other 'gotchas'. Something as seemingly straightforward as our use of the words 'and', 'or', 'not' and 'then' can catch us out. For example, the statement:

> 'Everyone whose surname begins with A and B is assigned to Group 1.'

might seem logical to a human, but to a computer, which strictly follows the logical meaning of AND, it's illogical. That's because, as written, the statement tests two things: does a surname begin with A and does it begin with B? However, no surname can have different first letters at the same time. You can ask a computer to carry out this test as much as you like, but it will always fail.

Furthermore, in everyday language, people sometimes use 'and' as a sequencing term, as in:

> 'The player took their turn and the game was finished.'

Again, that makes no logical sense to a computer. A logical statement like this describes a state of affairs at an instant in time. A player cannot make a move in a game when the game is finished.

To avoid confusion, never play loose with the phrasing of logic. Always use the correct operators. Truth tables are extremely helpful when in doubt.



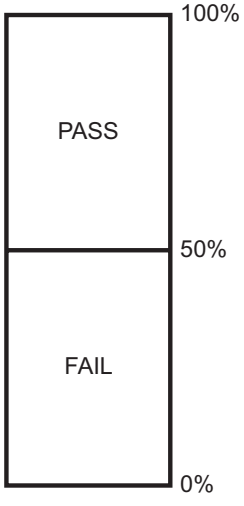
Logic and algorithms treat the 'if–then' form a little differently from each other. Consider 'if X, then Y'. Whereas a logical implication states how the truth values of X and Y are related, the intent of the algorithmic 'if–then' form is to make the execution of step Y dependent on X being true. If X isn't true, step Y isn't executed.

Missing certain eventualities

Another 'gotcha' related to the use of 'if–then' in algorithms is what to do when the condition is false. For example, here's an algorithm used to grade a student's work. It's

very simple. A student can earn one of two grades (pass or fail) and the student requires a score over 50 per cent to gain a pass (see Figure 2.6).

Figure 2.6 Grading levels for fail and pass



If score is greater than 50 per cent,
then mark student's grade as a pass.

Do you see a problem?

What happens in cases when the student scores 50 per cent or less? Common sense tells a human that the student should get a failing grade, because that's what the problem implies. But remember, computers have no common sense and won't do anything you don't tell them to do. That's why algorithms also include the 'if–then–else' form.[12] Using this, you could amend the algorithm to look like this:

if score is greater than 50 per cent,
then mark student's grade as a pass,
else mark student's grade as a fail.

When the condition in an 'if–then–else' is true, only the part after the 'then' is executed; when the condition is false, only the part after the 'else' is executed.

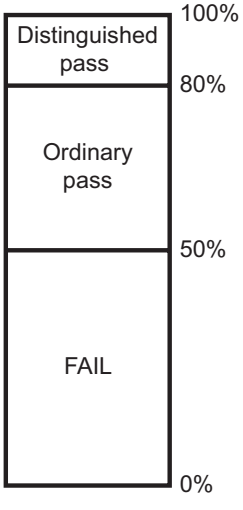
Complex conditionals

By itself, the NOT operator is simple to understand. For example, the grading algorithm could be rewritten as follows:

if score is not greater than 50 per cent,
then mark student's grade as a fail,
else mark student's grade as a pass.

No surprises there. However, difficulty can arise when a condition becomes more complex. Imagine an additional grade is introduced: students scoring over 80 per cent are awarded a distinguished pass (see Figure 2.7). A pass would only be awarded to scores over 50 and less than 80 per cent.

Figure 2.7 Grading levels for fail, ordinary pass and distinguished pass



To check for an ordinary passing score, you might begin an 'if–then' that tests two conditions like this:

```
if score is not less than 51 per cent or greater than 80 per cent,
then mark student's grade as an ordinary pass
...
```

A deliberate mistake lies in this little algorithm. All will be revealed shortly.



That might seem precise at first glance, but it could be interpreted in at least two different ways. Specifically, does the 'not' apply to both conditions or only to the first? Taking everyday language as a guide suggests it applies to both, just like saying 'the car is not big or red' means the car is neither big nor red. However, the realms of natural language and logic don't always align. This is a case where they don't and we'll see why shortly.

To avoid such confusion, symbolic logic provides very specific rules about how to interpret the various parts of logical statements. These rules are called the **order of precedence**. They dictate in what order each part of a logical expression should be evaluated. It all works just like in mathematics.

Table 2.8 Order of precedence for selected logical operators

Rank	Operator	Name
1	( )	Parentheses
2	¬	Not
3	>	Greater than
	<	Less than
4	Λ	Logical AND
5	V	Logical OR

When evaluating something like

$$9 - 3 \times 4$$

you get different answers depending on the order in which you carry out the subtraction and the multiplication. But strictly speaking, there's only one correct answer. That's because the rules of the order of precedence dictate in which order each part is evaluated; those with higher precedence are carried out first. To make the order explicit, you can put brackets around each expression. If we were to put them in explicitly, it would look like this:

First the multiplication

$$9 - (3 \times 4)$$

then the subtraction

$$(9 - (3 \times 4))$$

Technically, that means exactly the same thing as the original expression. Now we can see clearly that the multiplication must happen first and so the correct answer is -3. If you wanted to perform the subtraction first, that would be fine, but you would need to put your own brackets into the expression.[13] In this case, it would appear so:

$$(9 - 3) \times 4$$

This changes the meaning of the expression and its result (24).

Let's return to our student grading example. Before applying the rules, we should rewrite the expression using correct notation. Taking each condition one by one:

score is not less than 50 per cent
becomes
¬ score < 51

while
    greater than 80 per cent
becomes
    score > 80

Connecting them with the proper OR symbol gives us:

$$\text{if} \neg \textit{score} < 51 \text{V} \textit{score} > 80$$

To help us see the order in which things are evaluated, put the implicit brackets in place. Negation has the highest precedence of the operators here, so it gets brackets first:

$$\text{if} \neg (\textit{score} < 51) \text{ V } \textit{score} > 80$$

Comparisons have a higher precedence than disjunctions, so they get their brackets next (the left-hand comparison already has brackets thanks to the negation operator):

$$\text{if} \neg (\textit{score} < 51) \text{ V } (\textit{score} > 80)$$

Finally, the disjunction gets its brackets.

$$\text{if}(\neg (\textit{score} < 51) \text{ V } (\textit{score} > 80))$$

The expression is ready to be evaluated. To do that, we need to assign a value to 'score'. Let's arbitrarily choose 55.

Now that the brackets are in place, we can go from left to right, evaluating each piece as we go. NOT flips the truth value of the thing to its right, so we need to work out if that thing is true or not. 55 is not less than 51, therefore we substitute 'score < 51' with its truth value:

$$\text{if} \neg (\textit{false}) \text{ V } (\textit{score} > 80)$$

In other words:

$$\text{if } \textit{true} \text{ V } (\textit{score} > 80)$$

We've worked out the value of OR's left-hand condition. Now we need to evaluate the condition on its right. 55 is not greater than 80, therefore:

$$\text{if } \textit{true} \text{ V } \textit{false}$$

Consulting the truth table for OR, we find that when either or both conditions are true, then the whole expression is true. In this case, the student correctly gets an ordinary pass.

Let's try another score to make sure the expression works for distinguished passes too. We'll use 85 as the score. 85 is not less than 51, therefore:

$$\text{if} \neg (\textit{false}) \text{ V } (\textit{score} > 80)$$

In other words:

if *true* V (*score* > 80)

And 85 is greater than 80, so:

if *true* V *true*

Again, we've evaluated the expression to find that it is true. But this is not what we wanted. Look again at the expression in the context of the algorithm:

> if score is not less than 51% or greater than 80%,
> then mark student's grade as an ordinary pass,
> ...

We've demonstrated that, according to the algorithm in its present state, the student gets an ordinary pass even though they scored over 80!

The problem is that our natural language way of expressing our intentions doesn't quite agree with the rules of logic. To correct our original algorithm, we would have to put our own explicit brackets into the expression to 'override' the usual orders of precedence, like so:

if¬(*score* < 51 V *score* > 80)

This time, we evaluate **everything** inside the brackets first before evaluating the negation. Now, for a score of 85, the procedure looks like this:

if¬(*score* < 51 V *score* > 80)
if¬(*true* V *score* > 80)
if¬(*true* V *true*)
if¬(*true*)
if *false*

The student will therefore not be assigned the wrong grade, which is good because it's always embarrassing when that happens.



To avoid mistakes like this, it's a good idea to use the correct logical notation from the very start, even when you're just beginning to sketch out your ideas.

SUMMARY

Logic and systematic reasoning are key underpinnings of CT because computer programs are essentially an automation of our reasoning processes. Therefore, writing solutions requires a good understanding of logic and the ability to use proper notation over natural language. This chapter has covered the basics of inductive and deductive reasoning, as well as the importance of Boolean and symbolic forms of logic to computing.

Logic is an essential part of algorithms, which are the among the bedrock of computing. They are the core of any program and thus play a part in every computer-based solution. This chapter introduced the basic building blocks of every algorithm and the notion of state.

It's important in the case of both logic and algorithms to respect the need for a systematic approach. Although both can make intuitive sense on some level, it's also easy to make mistakes when considering them that way. The use of correct logical notation will help you to avoid making common mistakes.

EXERCISES

EXERCISE 1

Mark the following statements as true or false:

A. An inductive logical argument makes conclusions that are probable rather than certain.
B. So long as a deductive argument has true premises, then its conclusion is certain.
C. In Boolean logic, a logical expression can have a maximum of two propositions.
D. Logical AND has a higher precedence than logical OR.
E. $x = x + 1$ would be a valid expression in an algorithm.

EXERCISE 2

A recipe database stores the recipes in Table 2.9.

Users can search for recipes by entering search terms, which the database matches to tags and cooking times. Consider the following search terms and decide which recipes will be returned:

A. cooking time less than 20 minutes and not vegetarian;
B. includes chicken or turkey but not garlic;
C. doesn't include nuts.

Table 2.9 Recipes in the database

Name	Tags	Cooking time
Broiled chicken salad	Chicken, lettuce, gorgonzola cheese, lemon juice	15 mins
Holiday turkey	Turkey, rice, onion, walnuts, garlic	60 mins
Three-spice chicken	Chicken, ginger, cinnamon, garlic, green beans	30 mins
Lentil salad	Lentils, onion, peppers, walnuts, lettuce	20 mins
Garlic dip	Garlic, lemon juice, chickpeas, chicken broth	5 mins

EXERCISE 3

Here are some rules of thumb to apply when deciding which supermarket queue to join (assuming you want to spend as little time queuing as possible):

A. People in a queue who are carrying their items by hand take about one minute to be processed.
B. People carrying a basket take about two minutes to be processed.
C. People pushing a trolley take about five minutes to be processed for a half-empty trolley; ten minutes for a full trolley.
D. People in a self-service checkout are processed in about 80 per cent of the time of a normal checkout.

Express these rules of thumb as logical statements. Each statement should make a conclusion about the estimated queuing time.

EXERCISE 4

Take the logical statements from the previous question and incorporate them into an algorithm. The algorithm should take a queue as input, and should output the total estimated queueing time.

EXERCISE 5

Based on the algorithm for singing *99 Bottles of Beer*, write a pseudocode algorithm for singing *The 12 Days of Christmas*.

CHAPTER 1

Getting Started with Python

The Analytical Engine weaves algebraical patterns, just as the Jacquard loom weaves flowers and leaves.

— Ada, Countess of Lovelace, 1815–1853

1.1 ALGORITHMS AND ALGORITHMIC THINKING

The goal of this tutorial is to get you started in computational science using the computer language Python. Python is open-source software. You can download, install, and use it anywhere. Many good introductions exist, and more are written every year. *This* one is distinguished mainly by the fact that it focuses on skills useful for solving problems in physical modeling.

Modeling a physical system can be a complicated task. Let's take a look at how we can use the powerful processors inside your computer to help.

1.1.1 Algorithmic thinking

Suppose that you need to instruct a friend how to back your car out of your driveway. Your friend has never driven a car, but it's an emergency, and your only communication channel is a phone conversation before the operation begins.

You need to break the required task down into small, explicit steps that your friend understands and can execute in sequence. For example, you might provide your friend the following set of instructions:

```
1 Put the key in the ignition.
2 Turn the key until the car starts, then let go.
3 Push the button on the shift lever and move it to "Reverse."
4 ...
```

Unfortunately, for many cars this "code" won't work, even if your friend understands each instruction: It contains a **bug**. Before step 3, many cars require that the driver

```
Press down the left pedal.
```

Also, the shifter may be marked `R` instead of `Reverse`. It is difficult at first to get used to the high degree of precision required when composing instructions like these.

Because you are giving the instructions in advance (your friend has no mobile phone), it's also wise to allow for contingencies:

```
If a crunching sound is heard, press down on the left pedal ...
```

Breaking the steps of a long operation down into small, explicit substeps and anticipating contingencies are the beginning of *algorithmic thinking*.

If your friend has had a lot of experience watching people drive cars, then the instructions above may be sufficient. But a friend from Mars—or a robot—would need much more detail. For example, the first two steps may need to be expanded to something like

```
Grab the wide end of the key.
Insert the pointed end of the key into the slot on the lower right side
       of the steering column.
Rotate the key about its long axis in the clockwise direction
       (when viewed from the wide end toward the pointed end).
...
```

A *low-level* computer program provides instructions analogous to these in a language that a machine can understand.[1] A *high-level* system understands many common tasks, and therefore can be programmed in a more condensed style, like the first set of instructions above. Python is a high-level language. It includes commands for common operations in mathematical calculations, processing text, and manipulating files. In addition, Python can access many *standard libraries*, which are collections of programs that perform advanced functions like data visualization and image processing.

Python comes with a *command line interpreter* —a program that executes Python commands as you type them. Thus, in Python, you can type commands and execute them immediately. In contrast, many other programming languages used in scientific computing, like C, C++, or FORTRAN, require you to *compile* your programs before you can *execute* them. A separate program called a compiler translates your code into a low-level language. You then run the resulting compiled program to execute (carry out) your algorithm. With Python, it is comparatively easy to quickly write, run, and debug programs. (It still takes patience and practice, though.)

A command line interpreter combined with standard libraries and programs you write yourself provides a convenient and powerful scientific computing platform.

1.1.2 States

You have probably studied multistep mathematical proofs, perhaps long ago in geometry class. The goal of such a narrative is to establish the truth of a desired conclusion by sequentially appealing to given information and a formal system. Thus, each statement's truth, although not evident in isolation, is supposed to be straightforward in light of the preceding statements. The reader's "state" (list of propositions known to be true) changes while reading through the proof. At the end of the proof, there is an unbroken chain of logical deductions that lead from the axioms and assumptions to the result.

An **algorithm** has a different goal. It is a chain of instructions, each of which describes a simple operation, that accomplishes a complex task. The chain may involve a lot of repetition, so you won't want to supervise the execution of every step. Instead, you specify all the steps in advance, then stand back while your electronic assistant performs them rapidly. There may also be contingencies that cannot be known in advance. (`If a crunching sound is heard, ...`)

In an algorithm, the *computer* has a state that is constantly being modified. For example, it has many memory cells, whose contents may change during the course of an operation. Your goal might

[1] Machine code and assembly language are low-level programming languages.

be to arrange for one or more of these cells to contain the result of some complex calculation once the algorithm has finished running. You may also want a particular graphical image to appear.

1.1.3 What does `a = a + 1` mean?

To get a computer to execute your algorithm, you must communicate with it in a language it understands. The commands used in computer programming can be confusing at first, especially when they contradict standard mathematical usage. For example, many programming languages (including Python) accept statements such as these:

```
1 a = 100
2 a = a + 1
```

In mathematics, this makes no sense. The second line is an assertion that is always false; equivalently, it is an equation with no solution. To Python, however, "=" is not a test of equality, but an instruction to be executed. These lines have roughly the following meaning:[2]

1. Assign the name `a` to an integer object with the value 100.
2. Extract the value of the object named `a`. Calculate the sum of that value and 1. Assign the name `a` to the result, and *discard* whatever was previously stored under the name `a`.

In other words, the equals sign instructs Python to change its *state*. In contrast, mathematical notation uses the equals sign to create a proposition, which may be true or false. Note, too, that Python treats the left and right sides of the command `x=y` differently, whereas in math the equals sign is symmetric. For example, Python will give an error message if you say something like `b+1=a`; the left side of an assignment must be a name that can be assigned to the result of evaluating the right side.

We do often wish to determine whether a variable has a particular value. To avoid ambiguity between assignment and testing for equality, Python and many other computing languages use a double equals sign for the latter:

```
1 a = 1
2 a == 0
3 b = (a == 1)
```

The above code again sets up a variable `a` and assigns it to a numerical value. Then it compares this numerical value with 0. Finally, it creates a second variable `b`, and assigns it a logical value (**`True`** or **`False`**) after performing another comparison. That value can be used in contingent code, as we'll see later.

Do not use = (assignment) when == (test for equality) is required.

This is a common mistake for beginning programmers. You can get mysterious results if you make this error, because both = and == are legitimate Python syntax. In any particular situation, however, only one of them is what you want.

[2] T2 Appendix D gives more precise information about the handling of assignment statements.

1.1.4 Symbolic versus numerical

In math, it's perfectly reasonable to start a derivation with "Let $b = a^2 - a$," even if the reader doesn't yet know the value of a. This statement defines b in terms of a, whatever the value of a may be.

If we launch Python and immediately give the equivalent statement, `b=a**2-a`, the result is an error message.[3] Every time we hit `<Return/Enter>`, Python tries to compute values for every assignment statement. If the variable `a` has not been assigned a value yet, evaluation fails, and Python complains. Other computer math packages can accept such input, keep track of the symbolic relationship, and evaluate it later, but basic Python does not.[4]

In math, it's also understood that a definition like "Let $b = a^2 - a$" will persist unchanged throughout the discussion. If we say, "In the case $a = 1, \ldots$" then the reader knows that b equals zero; if later we say, "In the case $a = 2, \ldots$" then we need not reiterate the definition of b for the reader to know that this symbol now represents the value $2^2 - 2 = 2$.

In contrast, a numerical system like Python *forgets* any relation between `b` and `a` after executing the assignment `b=a**2-a`. All that it remembers is the *value* now assigned to `b`. If we later change the value of `a`, the value of `b` will *not* change.[5]

Changing symbolic relationships in the middle of a proof is generally not a good idea. However, in Python, if we say `b=a**2-a`, nothing stops us from later saying `b=2**a`. The second assignment updates Python's state by discarding the value calculated in the first assignment statement and replacing it with the newly computed value.

1.2 LAUNCH PYTHON

Rather than reading about what happens when you type some command, try out the commands for yourself. From now on, you should have Python running as you read, try every snippet of code, and observe what Python does in response. For example, this tutorial won't show you any graphics or output. You must generate these yourself as you work through the examples.

> *Reading this tutorial won't teach you Python. You can teach* yourself *Python by working through all the examples and exercises here, and then using what you've learned on your own problems.*

Set yourself little challenges and try them out. ("What would happen if . . . ?" "How could I accomplish that?") Python is not some expensive piece of lab apparatus that could break or explode if you type something wrong! Just try things. This strategy is not only more fun than passively accumulating facts—it is also far more effective.

A complete Python programming environment has many components. See Table 1.1 for a brief description of the ones that we'll be discussing. Be aware that we use "Python" loosely in this guide. In addition to the language itself, Python may refer to a *Python interpreter*, which is a computer application that accepts commands and performs the steps described in a program. Python may also refer to the language together with common libraries.

---

[3] The notation `**` denotes exponentiation. See Section 1.4.2.

[4] The SymPy library makes symbolic calculations possible in Python. See Section 7.3.1.

[5] In math, the statement $b = a^2 - a$ essentially defines b as a *function* of a. We can certainly do that in Python by defining a function that returns the value of $a^2 - a$ and assigning that function the name `b`. The point is that this is *not* what "=" does.